Investigacion 3 - b

1. Patrón Plantilla - Patron de Comportamiento

La idea principal es que la clase padre define qué pasos se deben realizar y en qué orden, mientras que las clases hijas definen cómo se realiza cada paso particular. Esto permite reutilizar la estructura común de diferentes algoritmos evitando la duplicación de código.

Por ejemplo, diferentes servicios pueden seguir una misma estructura:

- validar()
- buscar()
- procesar()
- guardar()

Ejemplo de una clase plantilla genérica:

```
public abstract class ServicioTemplate<T> {

    public final void ejecutar(T objeto) {
        validar(objeto);
        procesar(objeto);
        guardar(objeto);
    }

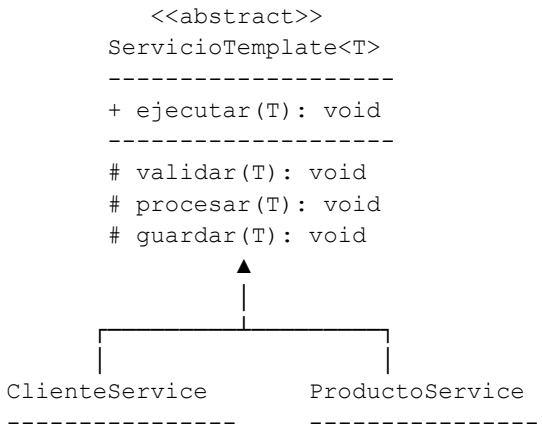
    protected abstract void validar(T objeto);

    protected abstract void procesar(T objeto);

    protected abstract void guardar(T objeto);
}
```

El método ejecutar() es el **método plantilla**. Normalmente se declara como final para impedir que las subclases modifiquen la estructura general del algoritmo. Las operaciones particulares quedan delegadas a los métodos abstractos.

Diagrama de clases



```
        # validar()          # validar()
        # procesar()         # procesar()
        # guardar()          # guardar()
```

¿Cómo utilizar Template Method en Controladores y Servicios?

En una aplicación Spring Boot podemos tener varios controladores con operaciones similares. Por ejemplo: ClienteController, ProductoController y PedidoController. Todos pueden seguir una estructura común: recibir la petición, validar los datos, ejecutar el servicio, procesar el resultado y devolver una respuesta.

```
public abstract class ControllerTemplate<T> {

    public ResponseEntity<?> ejecutar(T objeto) {

        validar(objeto);
        T resultado = procesar(objeto);

        return ResponseEntity.ok(resultado);
    }

    protected abstract void validar(T objeto);

    protected abstract T procesar(T objeto);
}
```

Un controlador concreto podría extender esta plantilla:

```
@RestController
public class ClienteController
        extends ControllerTemplate<Cliente> {

    @Override
    protected void validar(Cliente cliente) {
        // Validaciones específicas
    }

    @Override
    protected Cliente procesar(Cliente cliente) {
        // Lógica específica
        return cliente;
    }
}
```

Y otro controlador podría reutilizar la misma estructura:

```
@RestController
public class ProductoController
        extends ControllerTemplate<Producto> {

    @Override
    protected void validar(Producto producto) {
        // Validaciones específicas
    }
```

```
    @Override
    protected Producto procesar(Producto producto) {
        // Lógica específica
        return producto;
    }
}
```

2. Generics

¿Qué es Java Generics?

Generics es una característica de Java que permite crear clases, interfaces y métodos que trabajan con tipos parametrizados. En lugar de crear una clase específica para cada tipo, se puede definir una clase genérica cuyo tipo se determina posteriormente.

```
class Service<T> {
    ...
}
```

El parámetro T representa un tipo que será determinado al utilizar la clase. Por ejemplo, la misma clase puede utilizarse como Service<Cliente>, Service<Producto> o Service<Pedido>.

Ejemplo

Sin Generics, sería necesario crear servicios específicos:

```
public class ServicioCliente {

    public Cliente guardar(Cliente cliente) {
        // ...
        return cliente;
    }
}

public class ServicioProducto {

    public Producto guardar(Producto producto) {
        // ...
        return producto;
    }
}
```

Con Generics, se puede generalizar la operación:

```
public class Servicio<T> {

    public T guardar(T objeto) {
        // ...
        return objeto;
    }
}
```

```
Servicio<Cliente> clienteService = new Servicio<>();
Servicio<Producto> productoService = new Servicio<>();
```

Ventajas de Generics

Reutilización: Permite utilizar la misma clase o método para diferentes tipos.

Seguridad de tipos: El compilador puede detectar errores de tipos antes de ejecutar el programa.

Reduce conversiones: Evita muchas conversiones explícitas entre Object y tipos concretos.

Reduce código duplicado: Una única implementación puede utilizarse para diferentes entidades.

Mayor mantenibilidad: Los cambios en una implementación genérica pueden beneficiar a todas las clases que la utilizan.

Generics + Template Method

Ambos conceptos se complementan muy bien. Template Method permite generalizar la estructura del algoritmo, mientras que Generics permite generalizar el tipo de objeto sobre el cual trabaja.

```
public abstract class ServiceTemplate<T> {

    public final T ejecutar(T entidad) {

        validar(entidad);

        T resultado = procesar(entidad);

        guardar(resultado);

        return resultado;
    }

    protected abstract void validar(T entidad);

    protected abstract T procesar(T entidad);

    protected abstract void guardar(T entidad);
}
```

Servicio específico para clientes:

```
public class ClienteService
        extends ServiceTemplate<Cliente> {

    @Override
    protected void validar(Cliente cliente) {
        System.out.println("Validando cliente");
    }

    @Override
```

```
    protected Cliente procesar(Cliente cliente) {
        System.out.println("Procesando cliente");
        return cliente;
    }

    @Override
    protected void guardar(Cliente cliente) {
        System.out.println("Guardando cliente");
    }
}
```

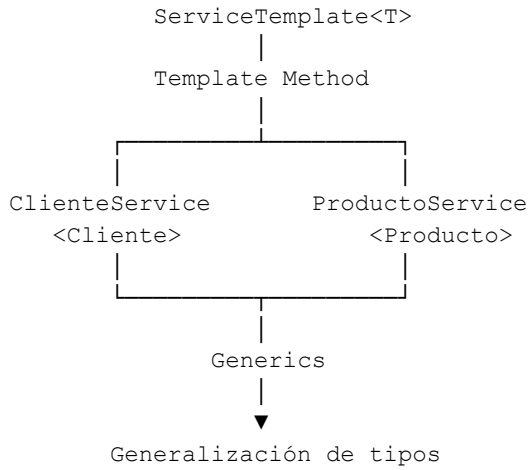
Servicio específico para productos:

```
public class ProductoService
        extends ServiceTemplate<Producto> {

    @Override
    protected void validar(Producto producto) {
        System.out.println("Validando producto");
    }

    @Override
    protected Producto procesar(Producto producto) {
        System.out.println("Procesando producto");
        return producto;
    }

    @Override
    protected void guardar(Producto producto) {
        System.out.println("Guardando producto");
    }
}
```



3. Reflection

¿Qué es Java Reflection?

Java Reflection es un mecanismo que permite que un programa pueda examinar y manipular clases, atributos, métodos y constructores durante la ejecución. Permite descubrir información de una clase dinámicamente y trabajar con elementos que normalmente no son accesibles directamente.

Mediante Reflection es posible obtener:

- atributos
- métodos
- constructores
- modificadores
- tipos
- anotaciones

Ventajas de Reflection

Permite trabajar dinámicamente con clases: Se puede determinar qué clase se está utilizando durante la ejecución.

Permite inspeccionar objetos: Es posible obtener sus atributos y métodos.

Es utilizada por frameworks: Frameworks como Spring utilizan Reflection para implementar funcionalidades como inyección de dependencias y procesamiento de anotaciones.

Permite construir herramientas genéricas: Es útil para frameworks, ORM, serializadores, sistemas de testing e inyección de dependencias.

Acceder a un atributo privado

Supongamos la siguiente clase:

```
public class Persona {

    private String nombre;

    public Persona(String nombre) {
        this.nombre = nombre;
    }
}
```

Normalmente no podemos acceder directamente a nombre porque es private. Con Reflection:

```
import java.lang.reflect.Field;

public class Main {

    public static void main(String[] args) throws Exception {
```

```
            Persona persona = new Persona("Diego");

            Class<?> clase = persona.getClass();

            Field campo = clase.getDeclaredField("nombre");

            campo.setAccessible(true);

            String nombre = (String) campo.get(persona);

            System.out.println(nombre);
        }
    }
```

Resultado:

```
Diego
```

Explicación de las instrucciones:

```
Class<?> clase = persona.getClass();
```

Obtiene el objeto Class que representa la clase Persona.

```
Field campo = clase.getDeclaredField("nombre");
```

Busca el atributo llamado nombre. getDeclaredField() permite obtener campos declarados en la clase, incluidos aquellos con acceso private.

```
campo.setAccessible(true);
```

Permite acceder al campo aunque tenga el modificador private.

```
campo.get(persona);
```

Obtiene el valor del atributo para la instancia persona.

Invocar un método mediante Reflection

Supongamos una clase con un método privado:

```
public class Persona {

    private String nombre;

    public Persona(String nombre) {
        this.nombre = nombre;
    }

    private void saludar() {
        System.out.println("Hola " + nombre);
    }
}
```

El método privado puede invocarse mediante Reflection:

```
import java.lang.reflect.Method;

public class Main {

    public static void main(String[] args) throws Exception {

        Persona persona = new Persona("Diego");

        Class<?> clase = persona.getClass();

        Method metodo = clase.getDeclaredMethod("saludar");

        metodo.setAccessible(true);

        metodo.invoke(persona);
    }
}
```

Resultado:

```
Hola Diego
```

Las instrucciones principales son:

```
Class<?> clase = persona.getClass();

Method metodo =
        clase.getDeclaredMethod("saludar");

metodo.setAccessible(true);

metodo.invoke(persona);
```

Método con parámetros

Reflection también permite invocar métodos que reciben parámetros:

```
public class Persona {

    private String nombre;

    public Persona(String nombre) {
        this.nombre = nombre;
    }

    private void saludar(String mensaje) {
        System.out.println(mensaje + " " + nombre);
    }
}
```

```
Method metodo =
        Persona.class.getDeclaredMethod(
                "saludar",
                String.class
        );
```

```
metodo.setAccessible(true);

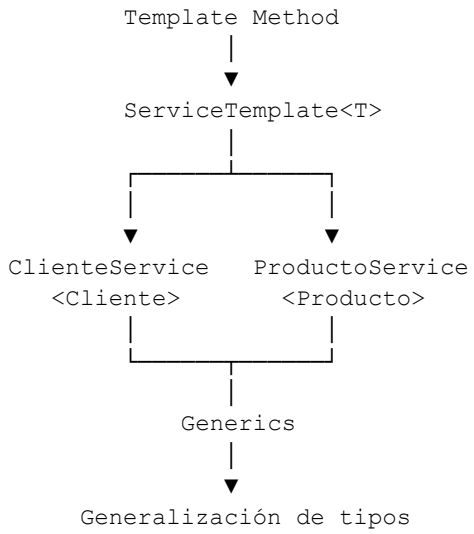
metodo.invoke(persona, "Hola");
```

Resultado:

```
Hola Diego
```

4. Relación entre los tres conceptos

Los tres conceptos pueden combinarse para construir una arquitectura más genérica. Template Method permite reutilizar el algoritmo, Generics permite reutilizarlo para diferentes tipos y Reflection permite trabajar con la estructura de las clases dinámicamente durante la ejecución.




```
Reflection puede utilizarse para descubrir
dinámicamente clases, métodos y atributos.
```

Por ejemplo, un framework podría detectar automáticamente una clase anotada con @Service y, mediante Reflection, inspeccionar sus métodos para determinar qué operaciones puede ejecutar.